

diffusionsde: conditional diffusion models for stochastic dynamical systems

Jack Griffiths^a, Matthew O. A. Ellis^{b,*}

^a*School of Computer Science, Sheffield University, United Kingdom*

^b*School of Computer Science, Sheffield University, United Kingdom*

Abstract

A stochastic dynamical system defines a probability distribution over trajectories. We present `diffusionsde`, a Python package that creates generative models of dynamical systems from simulated or measured ensembles using diffusion modelling [1, 2, 3]. The package supports ancestral DDPM sampling and reduced-step DDIM sampling. With $\eta = 0$ and fixed initial noise, the DDIM update can be made deterministic. Trajectories may be conditioned on initial conditions, external forcing, other time-dependent signals, or nothing at all. Once a model is learnt, it can be used to generate new trajectories given only the conditioning signal (e.g., to create a set of realistic trajectories given a set of initial conditions, or an external forcing waveform). The package also supports downstream tasks that rely upon the differentiability of the generative model, such as the inverse problem: given a set of trajectories, what was the conditioning signal that generated them? The package is designed to be agnostic to the underlying system, and it is therefore applicable to both simulated and measured data. We present three examples of increasing complexity to illustrate the package’s capabilities: an Ornstein–Uhlenbeck process, a stochastically forced Duffing oscillator, and two experimental magnetic-reservoir-computing systems (for which a trusted governing equation is not available).

Keywords: diffusion model, stochastic processes, dynamical systems, generative modelling, time series, inverse problems, Python

*Corresponding author.

Email addresses: jack.griffiths@sheffield.ac.uk (Jack Griffiths),
m.o.a.ellis@sheffield.ac.uk (Matthew O. A. Ellis)

1. Introduction

Many physical systems are intrinsically stochastic. Thermal fluctuations, environmental noise, unresolved degrees of freedom and random forcing mean that repeated experiments or simulations performed under nominally identical conditions need not produce the same trajectory. In such systems, the quantity of interest is therefore not generally a single time series, but a probability distribution over possible time series. Given a set of physical parameters or external controls $\mathbf{c}$, the modelling problem is to characterise the conditional trajectory distribution $p(\mathbf{x}_{0:T} \mid \mathbf{c})$, including its fluctuations, temporal correlations, and other statistical properties.

Numerical simulation provides a direct way to sample this distribution when the governing stochastic dynamics are known. It can, however, become expensive when large ensembles are required across many parameter values. This occurs routinely in uncertainty quantification, parameter studies, optimisation and inverse problems, where the simulator must be evaluated repeatedly. A deterministic surrogate is insufficient for these tasks: approximating a mean trajectory does not reproduce the distribution of fluctuations around it. This has motivated data-driven generative models for stochastic dynamics [4] [\[More refs needed\]](#). More generally, simulation-based inference has demonstrated the value of amortising expensive simulation into learned statistical models when conventional likelihood-based inference is impractical [5].

Diffusion models provide a direct way to learn distributions over trajectories without first identifying an explicit equation of motion. In a denoising diffusion probabilistic model (DDPM), clean training trajectories are progressively corrupted with Gaussian noise until they are nearly indistinguishable from random noise [1, 2]. During training, the neural network is given a noisy trajectory, the corresponding noise level, and any conditioning variables, and is trained to predict the noise that was added. Generation reverses this procedure: starting from Gaussian noise, the model repeatedly predicts and removes noise until a trajectory is obtained that follows the learned conditional distribution. The usual mean-squared-error noise-prediction objective is closely related to denoising score matching [6, 7]. Because the model acts on the complete time series, it learns the joint trajectory distribution rather than predicting each time point independently, allowing temporal correlations and stochastic variability to be retained.

We introduce `diffusionsde`, a Python library for constructing conditional diffusion models of stochastic dynamical systems. A model is trained from observed trajectories and optional conditioning variables and learns an approxima-

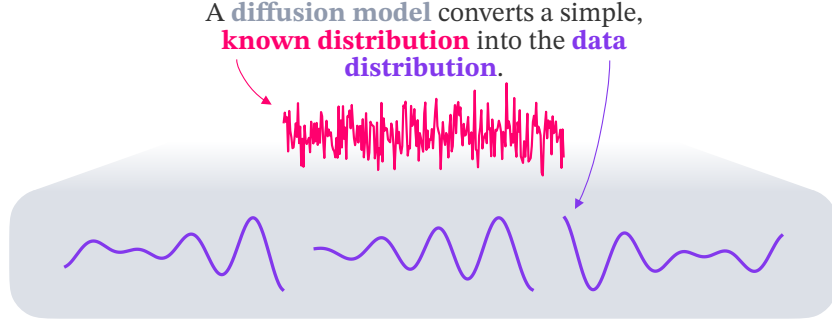


Figure 1: Conditional diffusion model for stochastic trajectories. Training samples a clean trajectory, adds Gaussian noise at a selected diffusion level, and trains the denoiser to predict the added noise. Generation starts from Gaussian noise and applies the trained denoiser repeatedly to obtain a trajectory conditioned on $\mathbf{c}$. The diffusion index k is distinct from the physical-time coordinate of the trajectory.

tion to their conditional distribution, $p_\theta(\mathbf{q}_{0:T} \mid \mathbf{c})$. After training, it can generate ensembles of trajectories within seconds for supplied conditions without rerunning the original stochastic simulator. The package implements denoising diffusion probabilistic model (DDPM) sampling and denoising diffusion implicit model (DDIM) sampling [3]. The library also exploits the differentiability of the generative model to support downstream tasks such as the inverse problem: given a set of trajectories, what was the conditioning signal that generated them?

The paper is split into X.

2. Problem formulation

2.1. Statement of the physical problem

Consider a stochastic dynamical system with state $\mathbf{x}(t) \in \mathbb{R}^m$ governed by

$$d\mathbf{x}_i = f_i(\mathbf{x}, t; \boldsymbol{\lambda}) dt + \sum_{j=1}^{m_W} G_{ij}(\mathbf{x}, t; \boldsymbol{\lambda}) dW_j, \quad i = 1, \dots, m, \quad (1)$$

or, in vector form, $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t; \boldsymbol{\lambda}) dt + \mathbf{G}(\mathbf{x}, t; \boldsymbol{\lambda}) d\mathbf{W}$. Here $\mathbf{f}$ is the drift, $\boldsymbol{\lambda}$ denotes physical parameters, $\mathbf{G} \in \mathbb{R}^{m \times m_W}$ is the diffusion matrix, and $\mathbf{W}(t) \in \mathbb{R}^{m_W}$ contains m_W independent Wiener processes. State-independent $\mathbf{G}$ gives additive noise and state-dependent $\mathbf{G}$ gives multiplicative noise. The examples in this paper use additive noise. Coloured noise can be included by augmenting the state with an additional stochastic variable, as in the Duffing oscillator of Section 3.2.

The package does not attempt to infer $\mathbf{f}$ or $\mathbf{G}$ (which is a divergence from e.g., a neural SDE). Instead, it learns the distribution of sampled trajectories produced by the stochastic system. The generated quantity is an approximation of a continuous trajectory $q(t)$. Sampling it at N_T physical-time points gives the vector representation $\mathbf{q}^{(s)} = (q_0^{(s)}, \dots, q_{N_T-1}^{(s)}) \in \mathbb{R}^{N_T}$, where $s = 1, \dots, N_S$, for an ensemble of N_S realisations.

Each trajectory may also be associated with a conditioning vector $\mathbf{c}^{(s)} \in \mathbb{R}^{N_T}$. The conditioning vector has the same length as the trajectory because it is supplied to the denoiser alongside the trajectory. It may represent a constant physical parameter, initial conditions stored in its leading entries and padded with zeros, or a time-dependent driving signal. If no physical conditioning is required, an empty conditioning vector can be used.

The physical system therefore defines a conditional trajectory distribution $\mathbf{q} \sim p_{\text{data}}(\mathbf{q} \mid \mathbf{c})$. The model learns $p_{\phi}(\mathbf{q} \mid \mathbf{c}) \approx p_{\text{data}}(\mathbf{q} \mid \mathbf{c})$, where ϕ denotes the trainable network parameters.

2.2. Minimal package interface

Training data are stored as a PyTorch dataset containing trajectory–condition pairs:

```
from torch.utils.data import TensorDataset

train_dataset = TensorDataset(
    trajectories,
    conditioning,
)
```

We observe more reliable training and inference when trajectories and conditioning variables are standardised. The library provides the corresponding standardisation and inverse-standardisation utilities.

The complete model interface is

```
from diffusionsde import DDPM

model = DDPM() # assuming default parameters are used
history = model.fit(train_dataset)
samples = model.generate(conditioning)
```

The remainder of this section is structured around these two operations. In

`model.fit()`, Gaussian noise is added to a clean trajectory and the network learns to predict that noise from the corrupted trajectory, its diffusion index, and the condition. In `model.generate()`, the trained network is used repeatedly to transform Gaussian noise into a clean trajectory that looks similar to the training data.

2.3. Training with `model.fit`

2.3.1. Forward diffusion

Diffusion introduces an artificial index k that is distinct from physical time t . Where appropriate, we will refer to k as the ‘algorithmic time’. Let N_K be the number of diffusion steps, with $k = 0$ denoting clean data. At each step, β_k specifies the variance of the Gaussian noise introduced by that step. It is useful to define the related quantities $\alpha_k = 1 - \beta_k$, $\bar{\alpha}_k = \prod_{\ell=1}^k \alpha_\ell$, and $\bar{\alpha}_0 = 1$, where α_k is the fraction of signal variance retained in one forward step, while $\bar{\alpha}_k$ is the corresponding fraction retained after k steps. Consequently, $\sqrt{\bar{\alpha}_k}$ determines the contribution of the original trajectory after k steps and $1 - \bar{\alpha}_k$ gives the accumulated noise variance.

The implementation uses a literature-standard linear schedule

$$\beta_k = \beta_1 + \frac{k-1}{N_K-1} (\beta_{N_K} - \beta_1), \quad k = 1, \dots, N_K, \quad (2)$$

where $0 < \beta_1 \leq \beta_{N_K} < 1$. The forward transition is

$$\mathbf{q}_k = \sqrt{\alpha_k} \mathbf{q}_{k-1} + \sqrt{\beta_k} \boldsymbol{\varepsilon}_k, \quad \boldsymbol{\varepsilon}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (3)$$

Successive steps reduce the contribution from the original trajectory and increase the Gaussian component.

Because the forward process is Gaussian, its marginal distribution is known analytically:

$$p_{\text{fwd}}(\mathbf{q}_k | \mathbf{q}_0) = \mathcal{N}(\mathbf{q}_k; \sqrt{\bar{\alpha}_k} \mathbf{q}_0, (1 - \bar{\alpha}_k) \mathbf{I}). \quad (4)$$

A sample at any diffusion level can therefore be obtained directly as

$$\mathbf{q}_k = \sqrt{\bar{\alpha}_k} \mathbf{q}_0 + \sqrt{1 - \bar{\alpha}_k} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (5)$$

There is no need to evaluate the preceding $k - 1$ forward steps during training.

2.3.2. Noise-prediction objective

For each training pair $(\mathbf{q}_0, \mathbf{c})$, `model.fit` samples $k \sim \text{Unif}\{1, \dots, N_K\}$, where $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, constructs $\mathbf{q}_k$ using Eq. (5), and passes $(\mathbf{q}_k, k, \mathbf{c})$ to the denoiser. The network output $\boldsymbol{\varepsilon}_\phi(\mathbf{q}_k, k, \mathbf{c})$ is an estimate of the noise that was added at that denoising step.¹

The default training objective is

$$\mathcal{L}(\phi) = \mathbb{E} \left[\frac{1}{N_T} \|\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\phi(\mathbf{q}_k, k, \mathbf{c})\|_2^2 \right]. \quad (9)$$

The expectation is estimated over minibatches using a Monte Carlo estimate.

2.3.3. Denoising network

The denoiser is a one-dimensional residual U-Net [9]. A U-Net applies the same encoder–decoder structure used in autoencoders, but applied to a time series, with one-dimensional convolutions acting along physical time. Transformer denoisers are common in larger/modern diffusion models [10]; the implementation considered here uses a convolutional U-Net.

For a minibatch of size B , $\mathbf{q}_k, \mathbf{c} \in \mathbb{R}^{B \times 1 \times N_T}$. The trajectory and condition are concatenated along the channel dimension, giving an input of shape $B \times 2 \times N_T$.

The encoder processes this input at progressively lower temporal resolutions. Each downsampling operation reduces the number of physical-time positions and increases the number of feature channels. A bottleneck operates at the lowest resolution. The decoder then restores the original temporal resolution. Skip connections pass encoder features directly to the corresponding decoder levels. The final convolution produces $\boldsymbol{\varepsilon}_\phi(\mathbf{q}_k, k, \mathbf{c})$ with the same shape as the noisy trajectory.

¹The noise predictor is related to the score of the noisy trajectory distribution [8, 6]. From Eq. (4),

$$\nabla_{\mathbf{q}_k} \log p_{\text{fwd}}(\mathbf{q}_k | \mathbf{q}_0) = -\frac{\boldsymbol{\varepsilon}}{\sqrt{1 - \bar{\alpha}_k}}. \quad (6)$$

The conditional distribution of noisy trajectories is

$$p_k(\mathbf{q}_k | \mathbf{c}) = \int p_{\text{fwd}}(\mathbf{q}_k | \mathbf{q}_0) p_{\text{data}}(\mathbf{q}_0 | \mathbf{c}) d\mathbf{q}_0. \quad (7)$$

For the MSE objective, the optimal noise predictor is $\mathbb{E}[\boldsymbol{\varepsilon} | \mathbf{q}_k, \mathbf{c}]$, giving

$$-\frac{\boldsymbol{\varepsilon}_\phi(\mathbf{q}_k, k, \mathbf{c})}{\sqrt{1 - \bar{\alpha}_k}} \approx \nabla_{\mathbf{q}_k} \log p_k(\mathbf{q}_k | \mathbf{c}). \quad (8)$$

Thus MSE noise prediction also estimates the conditional score at each diffusion level.

The main computational unit is a residual block [11]. Each block contains two one-dimensional convolutions, group normalisation [12], and SiLU activation, $\text{SiLU}(x) = x\sigma(x)$. In simplified form,

```
def forward(self, x, time_emb):
    h = self.conv1(x)
    h = self.norm1(h)
    h = self.act(h)

    time_emb = self.time_mlp(time_emb)[: , : , None]
    h = h + time_emb

    h = self.conv2(h)
    h = self.norm2(h)
    h = self.act(h)

    return h
```

Diffusion-step embedding. The network must know the diffusion index because the required denoising correction depends on the noise level. The scalar index k is first mapped to a fixed D_T -dimensional sinusoidal representation. For $j = 0, \dots, D_T/2 - 1$,

$$\omega_j = \exp \left[-\frac{j \log(10^4)}{D_T/2 - 1} \right], \quad (10)$$

and

$$\mathbf{e}(k) = [\sin(k\omega_0), \dots, \sin(k\omega_{D_T/2-1}), \cos(k\omega_0), \dots, \cos(k\omega_{D_T/2-1})]. \quad (11)$$

This is the same form of sinusoidal encoding used for transformer positions [13]. It provides fixed features that vary over different scales in k .

The encoding is then passed through a learned multi-layer perceptron,

$$\boldsymbol{\tau}_k = \text{Linear}_{D_T} [\text{SiLU} (\text{Linear}_{2D_T} [\mathbf{e}(k)])], \quad (12)$$

with dimensions $D_T \rightarrow 2D_T \rightarrow D_T$.

The vector $\boldsymbol{\tau}_k$ is the learned diffusion-step embedding. Each residual block applies its own linear projection to $\boldsymbol{\tau}_k$ so that its dimension matches the number of feature channels in that block. The projected vector is added between the two convolutions. The diffusion index therefore affects every residual block of the U-Net.

Dilated convolutions. Residual-block convolutions may also be dilated. For a kernel of size three, an ordinary convolution at physical-time index n uses the inputs $n - 1$, n , and $n + 1$, whereas dilation d uses $n - d$, n , $n + d$. The same number of kernel parameters is used in both cases. Dilation therefore increases the temporal range represented by a convolution without increasing the number of trainable, convolution weights.

The dilation factor is specified independently at each U-Net level:

```
if dilation_schedule is None:
    dilation_schedule = [1] * n_layers

for i in range(n_layers):
    d = dilation_schedule[i]

    block = ResidualBlock(
        in_ch,
        out_ch,
        time_emb_dim,
        dilation=d,
    )
```

Dilation is used only in the residual blocks; the initial, downsampling, upsampling, and final convolutions use $d = 1$.

It is our empirical observation that very long time series—especially those with long-time-correlated noise—train more reliably when dilation is used, since the network can represent long-range correlations without requiring a large number of downsampling levels.

2.3.4. Model defaults

Training uses Adam [14], cosine learning-rate annealing [15], shuffled mini-batches, and clipping of the global gradient norm to unity. If validation data are supplied, the parameter set with the lowest validation loss is restored after training.

Table 1 lists the principal defaults for `DDPM()`.

Model checkpoints store the configuration and fitted network weights. `DDPM.load` restores these quantities for generation, using the checkpoint with the lowest validation loss (if available), as a form of early stopping.

2.4. Generation with `generate`

Training provides an estimate of the noise present in a trajectory at each diffusion level. Generation uses this estimate to remove noise in an iterative process.

Table 1: Selected DiffusionConfig defaults.

Group	Entry	Default
Diffusion	steps N_K	1000
Diffusion	(β_1, β_{N_K})	$(10^{-4}, 0.02)$
Data	trajectory length	250
Architecture	hidden width	128
Architecture	encoder levels	4
Optimisation	learning rate	10^{-4}
Optimisation	batch size, epochs	32, 50
Loss	form	MSE; Huber optional

From Eq. (5),

$$\mathbf{q}_{N_K} = \sqrt{\bar{\alpha}_{N_K}} \mathbf{q}_0 + \sqrt{1 - \bar{\alpha}_{N_K}} \boldsymbol{\varepsilon}. \quad (13)$$

The schedule is chosen such that $\bar{\alpha}_{N_K} \ll 1$, so the terminal distribution is approximately $\mathbf{q}_{N_K} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Generation therefore starts from Gaussian noise and repeatedly applies the denoiser.

2.4.1. DDPM sampling

At step k , the network predicts $\boldsymbol{\varepsilon}_\phi(\mathbf{q}_k, k, \mathbf{c})$. For the Gaussian forward process, the reverse transition is

$$p_\phi(\mathbf{q}_{k-1} \mid \mathbf{q}_k, \mathbf{c}) = \mathcal{N}\left(\mathbf{q}_{k-1}; \boldsymbol{\mu}_\phi(\mathbf{q}_k, k, \mathbf{c}), \tilde{\beta}_k \mathbf{I}\right), \quad (14)$$

with

$$\boldsymbol{\mu}_\phi(\mathbf{q}_k, k, \mathbf{c}) = \frac{1}{\sqrt{\alpha_k}} \left[\mathbf{q}_k - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \boldsymbol{\varepsilon}_\phi(\mathbf{q}_k, k, \mathbf{c}) \right], \quad (15)$$

and

$$\tilde{\beta}_k = \beta_k \frac{1 - \bar{\alpha}_{k-1}}{1 - \bar{\alpha}_k}. \quad (16)$$

The mean gives the deterministic part of the reverse update. A Gaussian term with variance $\tilde{\beta}_k$ is then added. The implementation is equivalent to

```

predicted_noise = self.model(x_t, t, conditioning)

mean = ( x_t - beta * predicted_noise / torch.sqrt(1.0 -
    alpha_cumprod) ) / torch.sqrt(alpha)

if t > 0:
    noise = torch.randn_like(x_t)
    x_t = ( mean + torch.sqrt(posterior_variance) * noise )
else:
    x_t = mean

```

An ancestral DDPM sample evaluates all N_K reverse levels. No additional noise is added in the final transition because $\tilde{\beta}_1 = 0$.

2.4.2. DDIM sampling

The package also implements denoising diffusion implicit model (DDIM) sampling [3]. DDIM uses the same trained network but can evaluate only a subset of the diffusion levels—requiring fewer U-Net evaluations. The selected diffusion levels are $\kappa_0 > \kappa_1 > \dots > \kappa_{S-1} \geq 1$, with $\kappa_S = 0$. The choice of S is problem-specific and should be treated as a hyperparameter.

At diffusion level κ_j , the predicted noise gives an estimate of the clean trajectory,

$$\hat{\mathbf{q}}_0 = \frac{\mathbf{q}_{\kappa_j} - \sqrt{1 - \bar{\alpha}_{\kappa_j}} \boldsymbol{\varepsilon}_\phi(\mathbf{q}_{\kappa_j}, \kappa_j, \mathbf{c})}{\sqrt{\bar{\alpha}_{\kappa_j}}}. \quad (17)$$

The next selected diffusion level is

$$\sigma_j = \eta \sqrt{\frac{1 - \bar{\alpha}_{\kappa_{j+1}}}{1 - \bar{\alpha}_{\kappa_j}}} \sqrt{1 - \frac{\bar{\alpha}_{\kappa_j}}{\bar{\alpha}_{\kappa_{j+1}}}}, \quad (18)$$

$$\mathbf{q}_{\kappa_{j+1}} = \sqrt{\bar{\alpha}_{\kappa_{j+1}}} \hat{\mathbf{q}}_0 + \sqrt{1 - \bar{\alpha}_{\kappa_{j+1}} - \sigma_j^2} \boldsymbol{\varepsilon}_\phi(\mathbf{q}_{\kappa_j}, \kappa_j, \mathbf{c}) + \sigma_j \mathbf{z}_j, \quad \mathbf{z}_j \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (19)$$

The first term contains the estimated clean trajectory. The second gives the noise component required at the next selected diffusion level. The final term adds new Gaussian noise when $\eta > 0$.

The DDIM reverse process permits deterministic sampling, i.e., $\eta = 0$, $\sigma_j = 0$ at every step. Different initial Gaussian samples still generate different trajectories. In all examples in this work, we use deterministic sampling.

We still train the model with the full $N_K = 1000$ diffusion levels, but use $S = 200$ DDIM levels during generation. The distinction is therefore that DDPM is a training algorithm and DDIM is a sampling algorithm.

With DDIM and $\eta = 0$, fixing both `initial_noise` and the condition makes the generated trajectory deterministic. This is used by the differentiable inverse method described in Section 2.5. Setting `require_grad=True` retains the automatic-differentiation graph through the DDIM sampler; `grad_steps` can restrict this graph to a subset of the reverse steps to reduce memory use in downstream tasks.

2.5. Differentiable inverse estimation

A trained diffusion model can also be used for parameter estimation. Consider an observed ensemble of trajectories generated by a stochastic system with an unknown scalar parameter p . If the diffusion model has been trained with p as a conditioning variable, we can vary p until the generated ensemble agrees with the observations.

The important point is that DDIM generation is differentiable with respect to the conditioning input. We therefore treat p as an optimisation variable, generate trajectories conditioned on its current value, calculate a loss between the generated and observed ensembles, and backpropagate that loss through the complete generation process. The fitted U-Net is not updated during this procedure; only p is optimised.

The inverse calculation uses deterministic DDIM generation. The model is first frozen,

```
model.freeze()
```

which disables gradients with respect to the U-Net parameters while retaining differentiation with respect to its inputs. For each optimisation run, the solver also draws an initial Gaussian ensemble once and holds it fixed throughout the optimisation.

With DDIM and $\eta = 0$, fixing the initial noise removes random variation between successive evaluations of the objective. Changes in the generated trajectories during optimisation are then only caused by changes in p , rather than by drawing a different initial Gaussian sample.

The inverse solver compares the generated and observed ensembles through their time-dependent means and variances. In the implementation this is

```
loss = moment_matching_loss(  
    generated,  
    observed,  
    mean_weight=mean_weight,  
    variance_weight=variance_weight,  
)
```

where `moment_matching_loss` computes the mean and variance across the ensemble at each physical-time point and applies a mean-squared error to the two resulting curves. The two contributions can be weighted independently. This loss does not compare temporal correlations or power spectra and therefore does not, by itself, test all properties of the trajectory distribution, and future work may extend this approach.

Because `generated` retains its automatic-differentiation graph, the derivative of the loss with respect to p is obtained by ordinary backpropagation:

```
optimiser.zero_grad()  
loss.backward()  
optimiser.step()
```

The gradient passes from the ensemble loss, through every generated trajectory, through the DDIM reverse steps and U-Net evaluations, and finally to the conditioning parameter p . Gradients are computed using Adam.

The complete calculation is provided by `InverseProblem`. For example,

```

from diffusionsde import (
    InverseProblem,
    InverseProblemConfig,
)

model.freeze()

inverse = InverseProblem(
    model,
    observed,
    condition_from_parameter,
    parameter_range=(p_min, p_max),
    config=InverseProblemConfig(
        n_iterations=100,
        lr=0.05,
        n_gen=64,
    ),
)

result = inverse.solve(
    n_runs=8,
    return_trajectories=True,
)

p_estimates = result.estimate

```

Here `observed` has shape `[n_observed, n_time]`. At each optimisation iteration the solver generates `n_gen` trajectories at the current value of p , computes the moment-matching loss, backpropagates through the generator, and updates p by gradient descent. Each run starts from an independently chosen parameter value and an independently drawn, but subsequently fixed, Gaussian ensemble.

The parameter may be optimised directly in physical units or through the `sigmoid_bounded` parameterisation. In the latter case the optimiser acts on an unconstrained variable that is mapped smoothly into the supplied interval $(p_{\min}, p_{\max})$. We observe more reliable convergence when using the sigmoid-bounded parameterisation. The solver can also return the parameter value with the lowest loss encountered during each run rather than simply the final iterate, as a form of early stopping.

Backpropagating through every DDIM step can require substantial memory.

The `grad_steps` option restricts automatic differentiation to a subset of the reverse steps:

```
config = InverseProblemConfig(
    n_iterations=100,
    n_gen=64,
    grad_steps=20,
)
```

This reduces memory use and computation time. `grad_steps` should be treated as a hyperparameter to be tuned for each problem. For example, the OU process of Section 3.1 uses `grad_steps~100`, while the Duffing oscillator of Section 3.2 uses `grad_steps~10`.

3. Ornstein–Uhlenbeck

We test the model on two stochastic systems. The Ornstein–Uhlenbeck process provides an analytically tractable example for which the stationary distribution and autocorrelation are known. The driven Duffing oscillator provides a nonlinear example with stochastic forcing and no closed-form trajectory distribution. In both cases the model is trained with `fit` and held-out trajectories are generated with `generate`.

3.1. Ornstein–Uhlenbeck process

The Ornstein–Uhlenbeck (OU) process [16] is

$$dq = -\theta q \, dt + \sqrt{2D} \, dW, \quad (20)$$

where $\theta > 0$ is the relaxation rate and $D > 0$ is the diffusion coefficient. Its stationary distribution and normalised autocorrelation are, respectively,

$$q \sim \mathcal{N}\left(0, \frac{D}{\theta}\right), \quad \rho(\tau) = \exp(-\theta|\tau|). \quad (21)$$

These results provide separate tests of the fluctuation amplitude and temporal correlation learned by the diffusion model.

The trajectories used for training are generated with Euler–Maruyama,

$$q_{n+1} = (1 - \theta\Delta t)q_n + \sqrt{2D\Delta t} \, z_n, \quad z_n \sim \mathcal{N}(0, 1). \quad (22)$$

At finite Δt , this discrete process has stationary variance and lag- h autocorrelation

$$\text{Var}(q_n) = \frac{2D\Delta t}{1 - (1 - \theta\Delta t)^2}, \quad \rho_\Delta(h) = (1 - \theta\Delta t)^h. \quad (23)$$

Both expressions converge to the continuous-time results in Eq. (21) as $\Delta t \rightarrow 0$.

The notebook samples $\theta \sim \mathcal{U}(0.8, 3.0)$ and $D \sim \mathcal{U}(0.1, 1.5)$. Each simulation uses $\Delta t = 5/256$, discards 1024 initial steps, and retains $N_T = 256$ samples. The training set contains 512 trajectories and the held-out set contains 128. Trajectories and the two conditioning parameters are standardised using statistics calculated from the training set. The model uses 1000 forward diffusion steps and 200 DDIM steps for generation.

The test set contains parameter values that are not used for fitting. For each held-out pair (θ, D) , one reference trajectory and one generated trajectory are obtained. The comparison is therefore made over the full set of held-out conditions rather than from repeated sampling at a single value of (θ, D) . We compare the generated and reference trajectories through their marginal distribution and autocorrelation, with Eqs. (21) and (23) providing analytical references. At large lags, the empirical autocorrelation is affected by the finite trajectory length and by estimating and subtracting the mean of each trajectory; the same estimator is therefore applied to the reference and generated ensembles.

3.2. Driven Duffing oscillator

The second test is a periodically driven Duffing oscillator with white and coloured stochastic forcing. Introducing the velocity v and an auxiliary Ornstein–Uhlenbeck process ξ for the coloured force gives

$$dq = v dt, \quad (24)$$

$$dv = [-\mu v - \alpha_D q - \beta_D q^3 + \gamma \cos(\Omega t) + \xi] dt + \sqrt{2\mu\Theta} dW_1, \quad (25)$$

$$d\xi = -\frac{\xi}{\tau_c} dt + \sqrt{\frac{2D_\xi}{\tau_c}} dW_2. \quad (26)$$

The Wiener processes W_1 and W_2 are independent. We use $\alpha_D = -1$, $\beta_D = 1$, $\mu = 0.3$, $\gamma = 0.5$, $\Omega = 1.2$, $\Theta = 0.05$, $\tau_c = 0.5$, and $D_\xi = 0.02$. With $\alpha_D < 0$ and $\beta_D > 0$, the deterministic restoring potential is a double well [17].

Initial positions and velocities are sampled independently as

$$q(0) \sim \mathcal{U}(-1.8, 1.8), \quad v(0) \sim \mathcal{U}(-1, 1), \quad \xi(0) = 0. \quad (27)$$

A stochastic Heun method with $\Delta t = 0.05$ generates trajectories with $N_T = 256$ samples. The dataset contains 512 training trajectories and 128 held-out trajectories.

Only the initial position $q(0)$ is supplied to the diffusion model as a condition. The initial velocity is sampled when the reference trajectory is generated but is not given to the model. The learned distribution is therefore

$$p_\phi(\mathbf{q} \mid q_0), \quad (28)$$

with the variation due to the unobserved initial velocity and the stochastic forcing represented by the generated distribution. This provides a more demanding test than reconstructing a trajectory from a fully specified initial state.

The position trajectories and the q_0 conditioning values are standardised using the training data. The diffusion model uses 1000 forward diffusion steps and a 200-step DDIM sampler. As in the OU test, one model trajectory is generated for each held-out condition.

Unlike the OU process, the driven Duffing system has no closed-form trajectory distribution against which the model can be compared. We therefore compare the reference and generated ensembles using the time-dependent mean and standard deviation, autocorrelation, power spectrum, and pooled position distribution. The Wasserstein distance provides an additional comparison of the pooled marginals. These quantities test different aspects of the generated process: the mean and variance test its time-dependent distribution, while the autocorrelation and power spectrum test its temporal structure.

Because the held-out set contains different values of q_0 and only one generated trajectory is drawn for each value, these results test performance across the held-out initial-condition distribution. A direct test of $p(\mathbf{q} \mid q_0)$ at one fixed initial position would instead require repeated reference and generated trajectories at the same q_0 .

4. Experimental systems

4.1. Introduction

The numerical examples above test the method on systems for which the underlying dynamics are specified. We now consider three magnetic systems with different data sources and dimensionalities. The domain-wall oscillator (DWO) data are obtained from stochastic micromagnetic simulations. The nanoring and

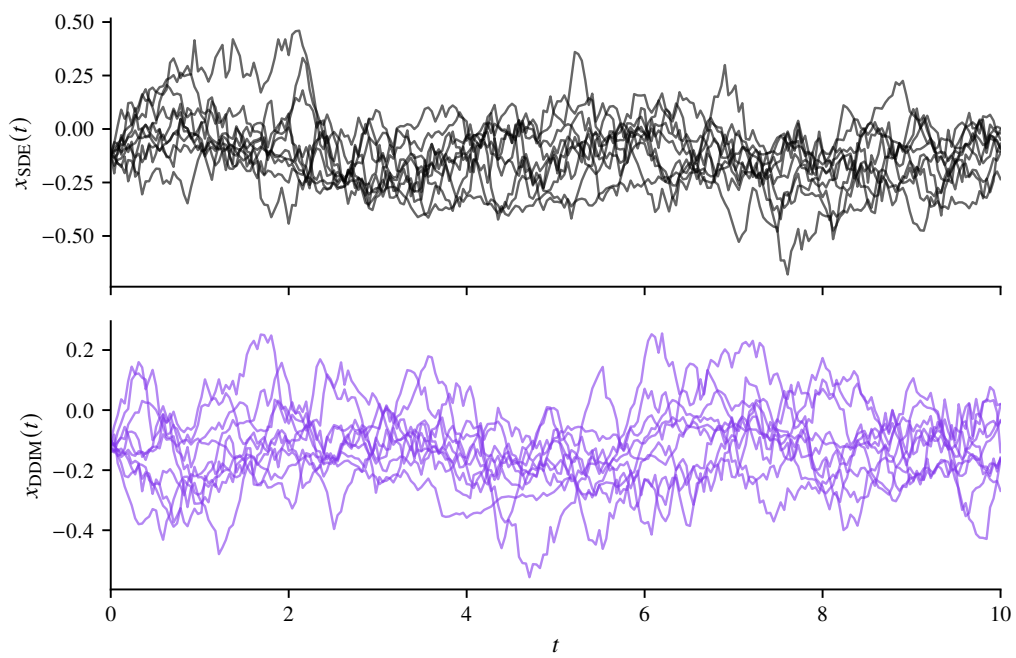


Figure 2: Sample trajectories from the Ornstein–Uhlenbeck system.

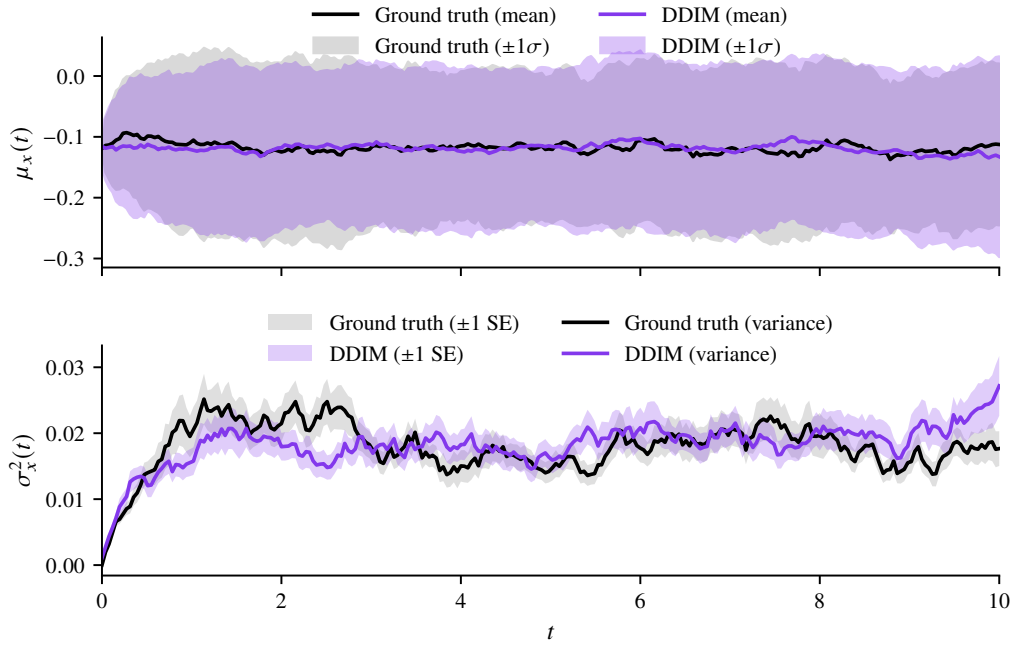


Figure 3: Comparison of statistics for the Ornstein–Uhlenbeck system.

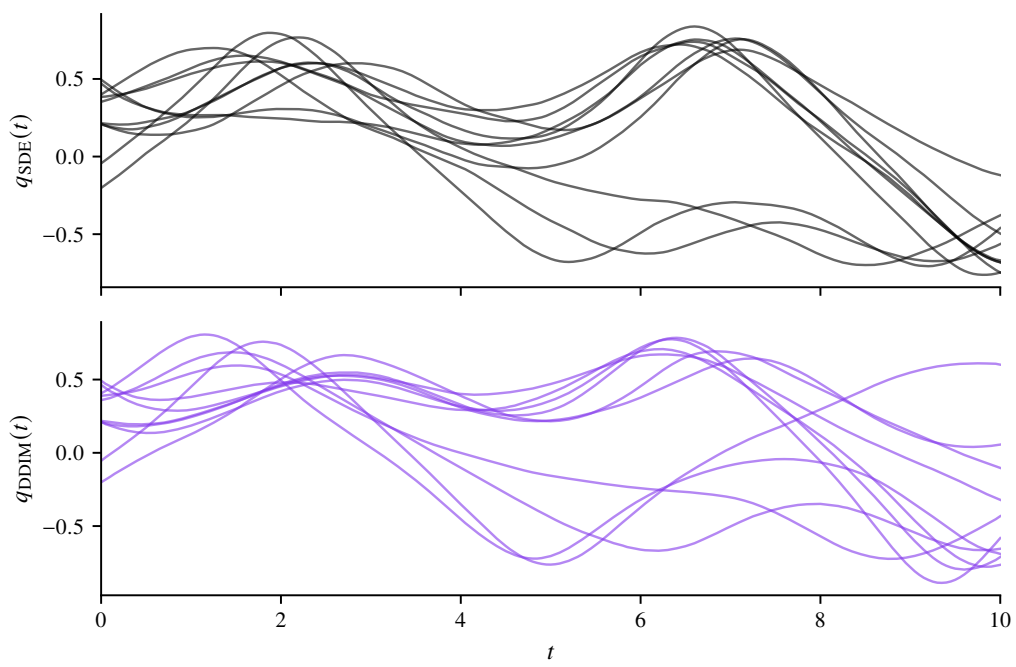


Figure 4: Sample trajectories from the Duffing oscillator system.

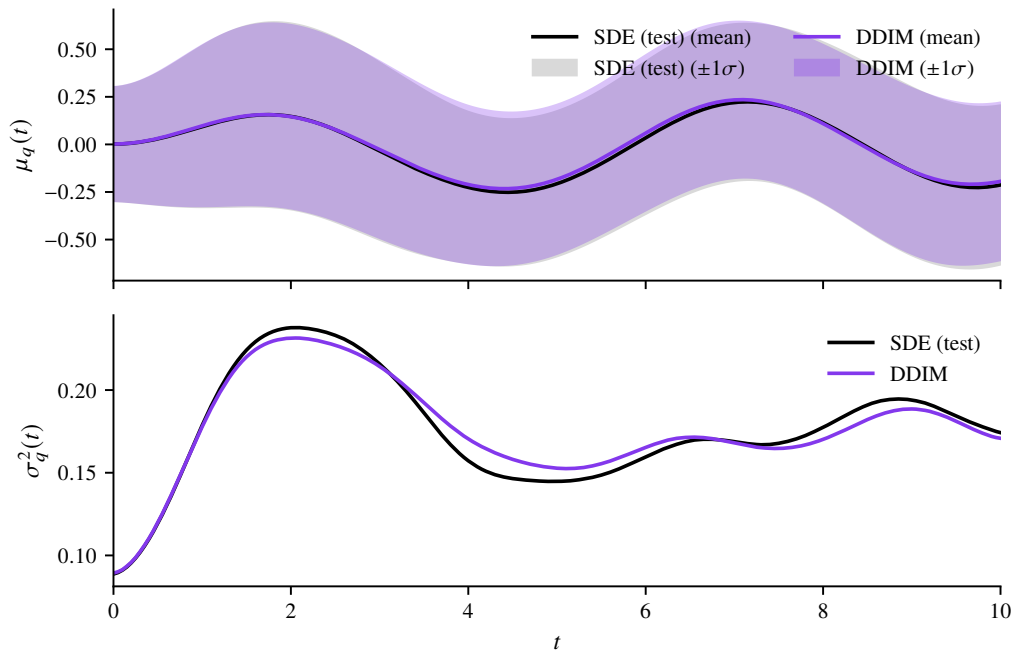


Figure 5: Comparison of statistics for the Duffing oscillator system.

artificial-spin-ice (ASI) data are experimental measurements. These cases test, respectively, surrogate modelling of a stochastic physical simulator, learning a conditional distribution from repeated measurements, and modelling a high-dimensional experimental response from a comparatively small dataset.

The diffusion procedure is unchanged between these systems. In each case, the model is trained by noise prediction and sampled using DDIM. The differences are confined to the observable, conditioning variables, preprocessing and trajectory representation.

4.2. Domain-wall oscillator

The first magnetic system is a confined domain wall driven by an oscillating magnetic field, of the type used for neuromorphic computation in Ref. [18]. The trajectories used here were generated by stochastic micromagnetic simulation using MuMAX3 [19]. The target variable is the x -component of the magnetisation, $m_x(t)$. Each trajectory is characterised by its initial conditions, the temperature T and the amplitude of the applied field H_0 . Each trajectory is conditioned on the temperature T and on the time-dependent applied field $B_x(t) = \mu_0 H_0 \sin(2\pi f t)$, where the frequency is fixed at $f = 499.995$ MHz. The conditioning tensor therefore contains one constant temperature channel and one time-dependent field channel. The diffusion model can in principle work with any number of conditioning channels.

Figure 7 compares the aggregate time-dependent statistics of the held-out micromagnetic and generated trajectories.

[Benchmarking...]

4.3. Magnetoresistance with full-waveform conditioning

The first dataset comprises magnetisation dynamics from an experimental spintronic device consisting of a lithographically patterned, interconnected array of permalloy $\text{Ni}_{80}\text{Fe}_{20}$ nanoscale rings. The array is driven by an in-plane rotating magnetic field with amplitude $H(t)$, and the resulting anisotropic magnetoresistance (AMR) is measured.² We refer the reader to Ref. [22] for details of the device and measurement procedure.

Each AMR channel is modelled separately. A training example is

$$\left(\mathbf{q}^{(i)}, \mathbf{h}^{(i)}\right), \quad \mathbf{q}^{(i)}, \mathbf{h}^{(i)} \in \mathbb{R}^{N_T}, \quad N_T = 5000,$$

²These dynamics provide the nonlinearity and fading memory required for reservoir computing [20, 21, 22].

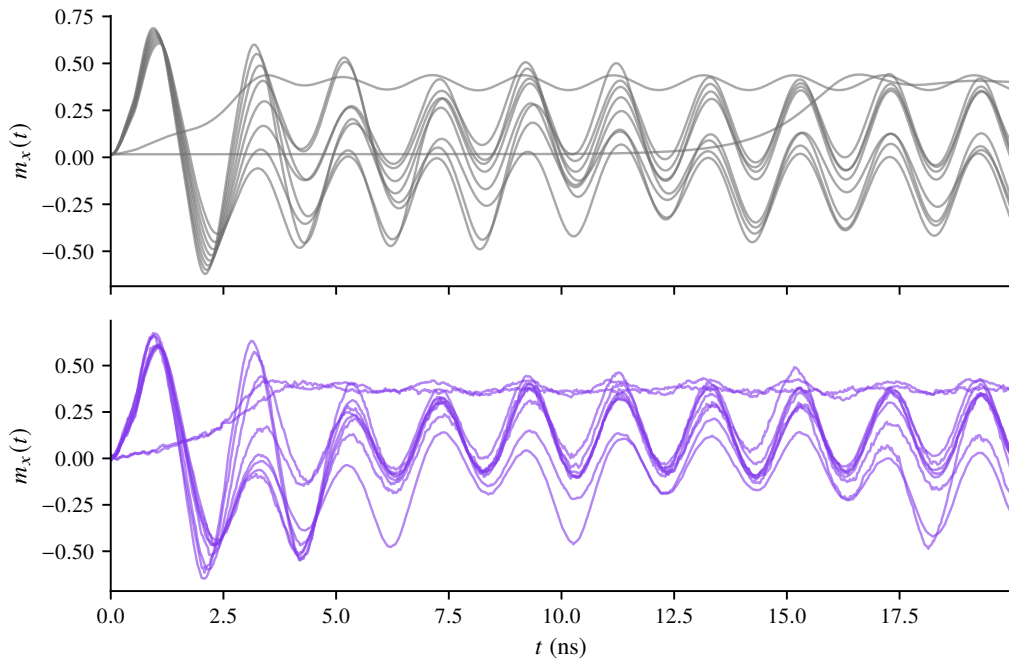


Figure 6: Sample trajectories from the domain-wall oscillator system.

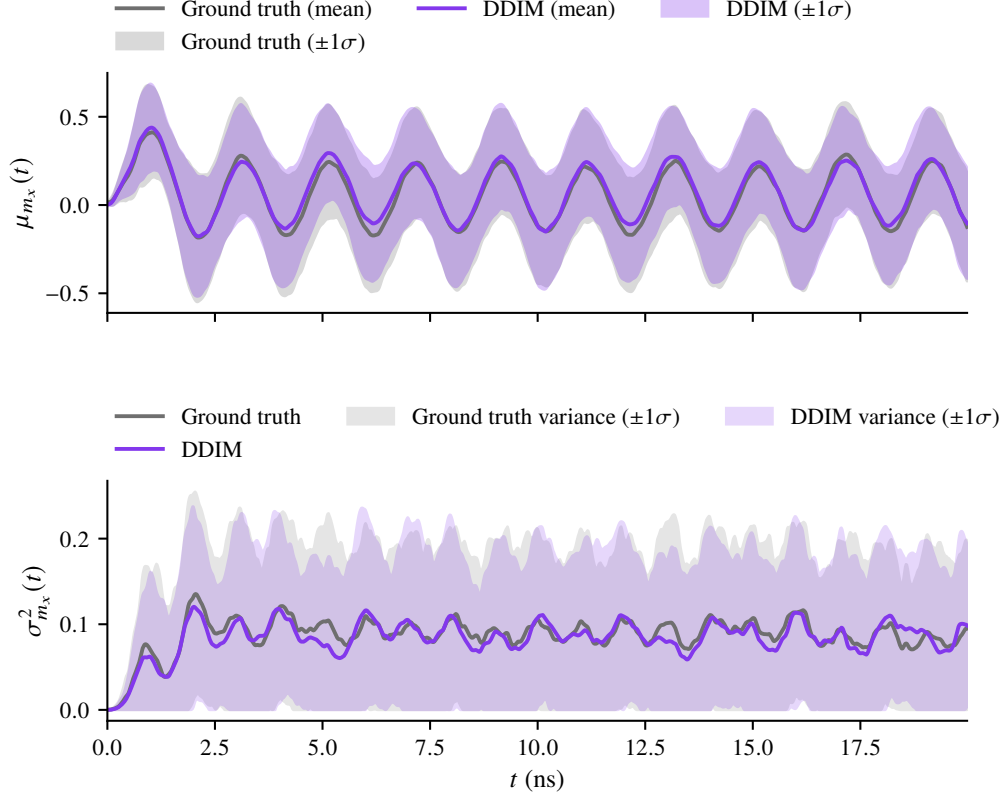


Figure 7: Held-out domain-wall-oscillator statistics. The time-dependent mean and spread of trajectories generated by the diffusion model are compared with the corresponding statistics of stochastic MUMAX3 trajectories. The evaluation uses 528 held-out trajectories and 100 DDIM sampling steps.

where $\mathbf{q}^{(i)}$ is the measured AMR trajectory and $\mathbf{h}^{(i)}$ is the corresponding discretised field-amplitude waveform. The raw measurements contain 100 repetitions of each of 999 field patterns. After removing acquisition transients, each trajectory contains 5000 samples, corresponding to 100 cycles of 50 samples at a sampling rate of 3200 Hz. The AMR is filtered using a fifth-order, zero-phase Butterworth band-pass filter from 40 to 200 Hz. The field amplitude is represented as a stepwise conditioning signal, with one normalised value held constant over each 50-sample cycle. Ten repetitions per pattern are used, and the patterns are divided into training, validation, and test sets in an 80:10:10 ratio.

The conditional U-Net has hidden width 64, kernel size 3, and dilation factors

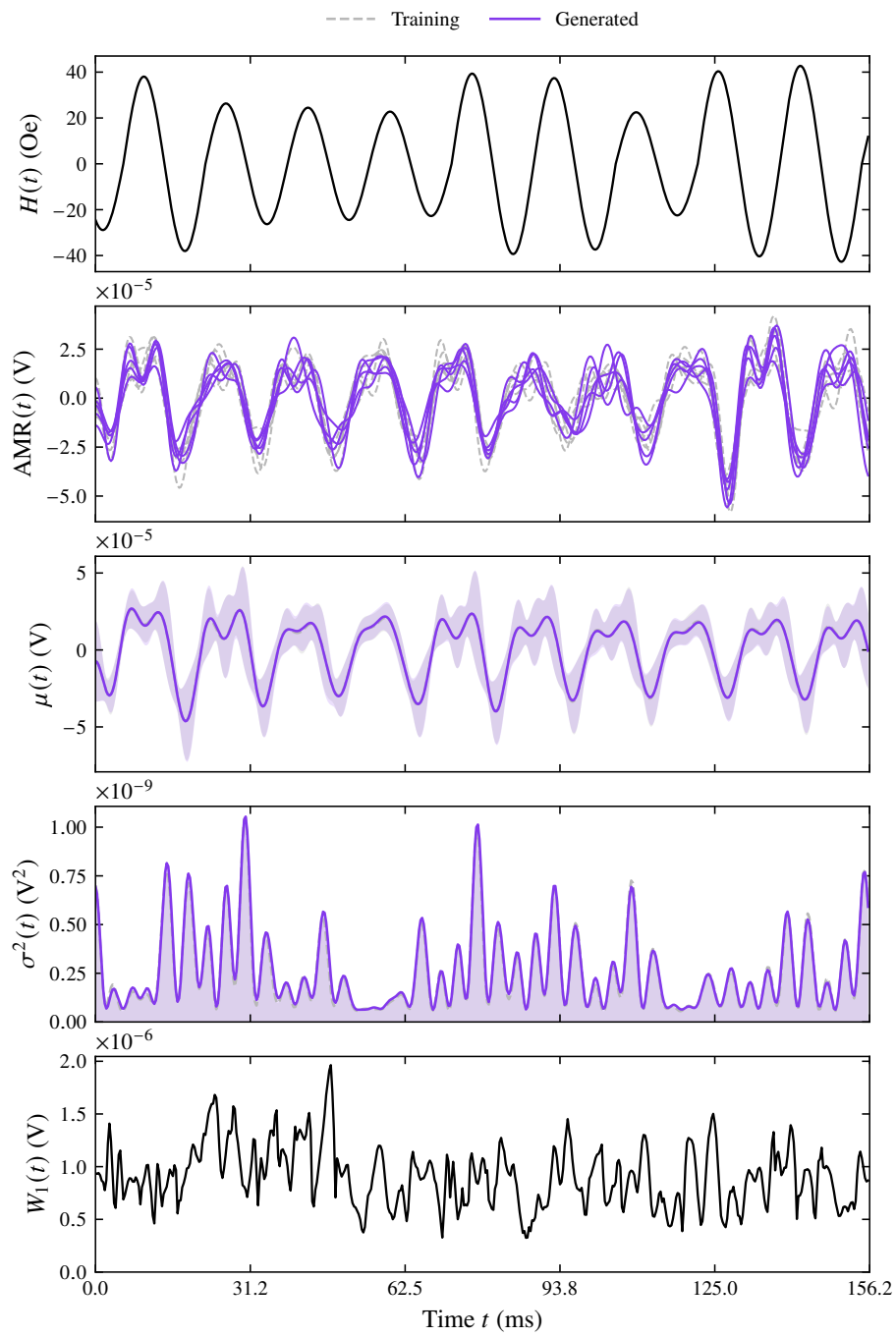


Figure 8:

(1, 2, 4, 8). The dilation schedule gives an effective receptive field of approximately 19 field cycles, or 329 ms, allowing the model to represent the long-range temporal correlations and coloured noise in the AMR signal. The trajectory length $N_T = 5000$ is independent of the 1000 algorithmic diffusion steps used during training. The model is trained with noise-prediction mean-squared error and sampled using DDIM.

4.4. Diffusion modelling of spectrograms: artificial spin ice

The second dataset is obtained from an artificial-spin-ice (ASI) multilayer reservoir probed by ferromagnetic resonance (FMR) spectroscopy [23]. The applied field, $H_{\text{app}}(t)$, varies between approximately 180 and 235 mT, following either a Mackey–Glass or sinusoidal waveform. The measured response is dP/dH , recorded in arbitrary units across $N_F = 201$ fixed frequency bins. Each trajectory is therefore a time–frequency matrix $\mathbf{Y}^{(i)} \in \mathbb{R}^{T \times 201}$. The data were separated into experimental runs, divided into $T = 50$ -step windows with stride 10, and split at the run level. This produced 359 training and 84 test windows for Mackey–Glass, and 448 training and 104 test windows for the sine wave. The field was globally z-scored, and each FMR channel was normalised using its training mean and standard deviation.

The data supplied to the diffusion model are multivariate one-dimensional time series. PCA was fitted to the training matrices after reshaping (from $[N_{\text{train}}, T, 201]$ to $[N_{\text{train}}T, 201]$), so that each time slice was one observation and the frequency bins were the features. After mode-wise normalisation, the trajectories were rearranged to shape $K \times T$. We use $K = 16$ modes, which explain 96.9% of the Mackey–Glass variance.

The diffusion model receives these PCA modes and is conditioned on the input field. A one-dimensional residual U-Net convolves along the time axis; the 16 PCA modes are treated as jointly generated channels. The U-Net has three levels of dilation: [1, 2, 4]. The PCA is then inverted to reconstruct the full spectrogram.

This representation is suited to the available data. The Mackey–Glass training set contains only 359 windows from 15 experimental runs, and the stride-10 windows overlap. A two-dimensional image model would learn directly in the $50 \times 201 = 10,050$ -value time–frequency space, with convolutions across both time and frequency. The PCA representation reduces the generated state to $16 \times 50 = 800$ values, which may capture the global correlations between the fixed frequency channels, and allows the U-Net to focus on temporal dynamics. It therefore provides a more data-efficient model for this small experimental dataset while retaining the full frequency response through inverse PCA. We observe that increasing

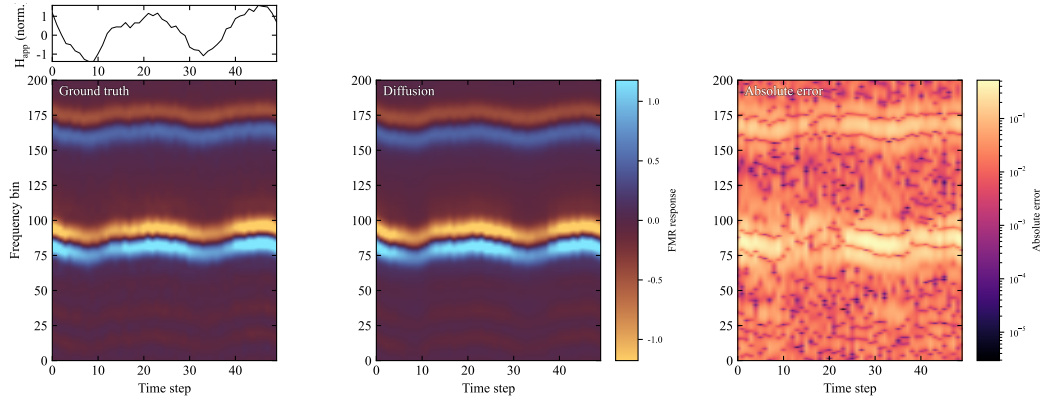


Figure 9:

the number of retained principal components beyond $K = 64$ deteriorates training, for reasons we do not yet understand.

Data availability

Selected case studies (Ornstein–Uhlenbeck and Duffing) are reproduced end to end by Jupyter notebooks distributed with the package. The experimental data are available upon request. Code, notebooks, and configuration files are available at <https://github.com/Neuromorphic-Spintronics/diffusion>.

References

- [1] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan, S. Ganguli, Deep unsupervised learning using nonequilibrium thermodynamics, in: *Proceedings of the 32nd International Conference on Machine Learning*, 2015, pp. 2256–2265.
- [2] J. Ho, A. Jain, P. Abbeel, Denoising diffusion probabilistic models, in: *Advances in Neural Information Processing Systems*, Vol. 33, 2020, pp. 6840–6851.
- [3] J. Song, C. Meng, S. Ermon, Denoising diffusion implicit models, in: *International Conference on Learning Representations*, 2021.
- [4] P. Kidger, J. Foster, X. Li, T. J. Lyons, Neural SDEs as infinite-dimensional GANs, in: *Proceedings of the 38th International Conference on Machine*

Learning, Vol. 139 of Proceedings of Machine Learning Research, PMLR, 2021, pp. 5453–5463.

- [5] K. Cranmer, J. Brehmer, G. Louppe, The frontier of simulation-based inference, *Proceedings of the National Academy of Sciences* 117 (48) (2020) 30055–30062. [doi:10.1073/pnas.1912789117](https://doi.org/10.1073/pnas.1912789117).
- [6] P. Vincent, A connection between score matching and denoising autoencoders, *Neural Computation* 23 (7) (2011) 1661–1674.
- [7] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole, Score-based generative modeling through stochastic differential equations, in: *International Conference on Learning Representations*, 2021.
- [8] A. Hyvärinen, Estimation of non-normalized statistical models by score matching, *Journal of Machine Learning Research* 6 (2005) 695–709.
- [9] O. Ronneberger, P. Fischer, T. Brox, U-Net: Convolutional networks for biomedical image segmentation, in: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, Springer, 2015, pp. 234–241.
- [10] W. Peebles, S. Xie, Scalable diffusion models with transformers, in: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4195–4205.
- [11] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [12] Y. Wu, K. He, Group normalization, in: *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 3–19.
- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: *Advances in Neural Information Processing Systems*, Vol. 30, 2017.
- [14] D. P. Kingma, J. L. Ba, Adam: A method for stochastic optimization, in: *International Conference on Learning Representations*, 2015.
- [15] I. Loshchilov, F. Hutter, SGDR: Stochastic gradient descent with warm restarts, in: *International Conference on Learning Representations*, 2017.

- [16] G. E. Uhlenbeck, L. S. Ornstein, On the theory of the Brownian motion, *Physical Review* 36 (5) (1930) 823–841.
- [17] J. Guckenheimer, P. Holmes, *Nonlinear Oscillations, Dynamical Systems, and Bifurcations of Vector Fields*, Springer, New York, 1983.
- [18] R. V. Ababei, M. O. A. Ellis, I. T. Vidamour, D. S. Devadasan, D. A. Allwood, E. Vasilaki, T. J. Hayward, Neuromorphic computation with a single magnetic domain wall, *Scientific Reports* 11 (2021) 15587. doi:[10.1038/s41598-021-94975-y](https://doi.org/10.1038/s41598-021-94975-y).
- [19] A. Vansteenkiste, J. Leliaert, M. Dvornik, M. Helsen, F. Garcia-Sanchez, B. Van Waeyenberge, The design and verification of MuMax3, *AIP Advances* 4 (10) (2014) 107133. doi:[10.1063/1.4899186](https://doi.org/10.1063/1.4899186).
- [20] R. W. Dawidek, T. J. Hayward, I. T. Vidamour, T. J. Broomhall, G. Venkat, M. A. Mamoori, A. Mullen, S. J. Kyle, P. W. Fry, N.-J. Steinke, J. F. K. Cooper, F. Maccherozzi, S. S. Dhesi, L. Aballe, M. Foerster, J. Prat, E. Vasilaki, M. O. A. Ellis, D. A. Allwood, Dynamically driven emergence in a nanomagnetic system, *Adv. Funct. Mater.* 31 (2021) 2008389. doi:[10.1002/adfm.202008389](https://doi.org/10.1002/adfm.202008389).
- [21] I. T. Vidamour, M. O. A. Ellis, D. Griffin, G. Venkat, C. Swindells, R. W. S. Dawidek, T. J. Broomhall, N. J. Steinke, J. F. K. Cooper, F. Maccherozzi, S. S. Dhesi, S. Stepney, E. Vasilaki, D. A. Allwood, T. J. Hayward, Quantifying the computational capability of a nanomagnetic reservoir computing platform with emergent magnetisation dynamics, *Nanotechnology* 33 (48) (2022) 485203. doi:[10.1088/1361-6528/ac87b5](https://doi.org/10.1088/1361-6528/ac87b5).
- [22] L. Manneschi, I. T. Vidamour, K. D. Stenning, C. Swindells, G. Venkat, D. Griffin, L. Gui, D. Sonawala, D. Donskikh, D. Hariga, E. Donati, S. Stepney, W. R. Branford, J. C. Gartside, T. J. Hayward, M. O. A. Ellis, E. Vasilaki, Noise-aware training of neuromorphic dynamic device networks, *Nat. Commun.* 16 (2025) 9192. doi:[10.1038/s41467-025-64232-1](https://doi.org/10.1038/s41467-025-64232-1).
- [23] K. Stenning, et al., Neuromorphic overparameterisation and few-shot learning in multilayer physical neural networks, *Nature Communications* 15 (2024). doi:[10.1038/s41467-024-50633-1](https://doi.org/10.1038/s41467-024-50633-1).